Functions: Reducing Duplication and Splitting Complex Tasks

Write logic once, name it, reuse it — and break big jobs into small, single-purpose steps.

Why functions?

- **DRY** – write the logic once and call it many times; fix a bug in one place.
- **Single responsibility** – one function, one job, named for what it does (`verb_noun`).
- **Testable & readable** – small functions can be tested alone and read like a story.

Anatomy of a function

```
# name + parameters (height has a default)
def area(width, height=1):
    """Return the area of a rectangle."""
    # hand a value back to the caller
    return width * height

print(area(3, 4))
```

OUTPUT
```
12
```

1 · Remove duplication

BEFORE — COPY-PASTED LOGIC
```
total_a = round(10 * (1 + 0.18), 2)
total_b = round(25 * (1 + 0.18), 2)   # same formula
# copy-pasted 3x
total_c = round(40 * (1 + 0.18), 2)
```

AFTER — ONE FUNCTION, MANY CALLS
```
def with_tax(price, rate=0.18):
    return round(price * (1 + rate), 2)

totals = [with_tax(p) for p in (10, 25, 40)]
print(totals)
```

OUTPUT
```
[11.8, 29.5, 47.2]
```

2 · Split a complex task

Break the big job into small steps; the top-level function only **orchestrates**.

```
# step 1: text -> data
def parse(line):
    name, score = line.split(",")
    return name.strip(), int(score)

# step 2: pure calculation
def average(scores):
    return sum(scores) / len(scores)

# orchestrator reads like a recipe
def report(lines):
    scores = [parse(l)[1] for l in lines]
    return f"avg={average(scores):.1f}"

print(report(["ann, 80", "bob, 90"]))
```

OUTPUT
```
avg=85.0
```

Refactoring recipe

- Spot repeated (or tangled) code.
- What **varies**? → make it a parameter.
- What is the **result**? → `return` it.
- Replace each copy with a call, then re-run your tests.

Good-function checklist

Rule	Why
Do one thing	easy to name, test and reuse
Name = verb phrase (`calc_tax`, `is_valid`)	reads like English
Return values, don't `print` them	the caller decides what to do
Keep it short (≈ 5–20 lines)	fits in your head
Avoid hidden inputs (globals)	same input → same output

TIP Rule of three: the third time you copy-paste a block, extract a function.

PITFALL Don't over-split. A function that only wraps one trivial line and is called once adds noise, not clarity.